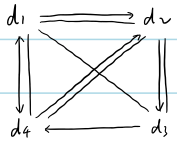
1.2



1.3 (1) $n-1$

(2) $n-1$, $n\geq 2$

1 , $n=1$

(3) 1 , $n\neq 2$

2 , $n=2$

(4) n

(5) $[\sqrt{n}]$

(6) 每10次 else $x\to 101$, 1次 if $x\to 91$, $y--$

$\Rightarrow$ 每11次 if-else y 減1, 共 $11\times 100=1100$ 次

(7) $n\to n-1\to\cdots\to 1\to n-1\to\cdots\to 1\to n-2\to\cdots\to 1\to\cdots\to 1$

$= n\cdot 1+(n-1)\cdot 2+\cdots+2\cdot(n-1)+1\cdot n$

$= \sum_{k=1}^{n} k(n-k+1)$

$= (n+1)\sum_{k=1}^{n} k - \sum_{k=1}^{n} k^2$

$= (n+1)\frac{(n+1)n}{2} - \frac{n(n+1)(2n+1)}{6}$

$= \frac{n(n+1)(n+2)}{6}$

2.3.

```
void InsertSqList ( SqList &La, ElemType x) {
    if (La.length ==La.listsize)     Increment (La);
    int i=La.length-1;
    while (i>=0 && La.elem[i] > x) {
        La.elem[i+1] = La.elem[i];
        i--; }
    La.elem[i+1] = x;
    La.length ++;     }
```

2.5.

```
void ReverseSqList ( SqList &L ) {
    if (L.length <=1)   return;
    int mid = L.length /2;
    for (int i=0; i<mid ; i++) {
        ElemType temp = L.elem[i];
        L.elem[i] = L.elem[L.length-1-i];
        L.elem[L.length-1-i] = temp; }
```

2.6.

```
void ReverseLinkList_WithHeader ( LinkList &L) {
    if (L->next == NULL || L->next->next ==NULL)     return;
    LNode *p = L->next;
    LNode *q;
    L->next = NULL;
    while (p != NULL) {
        q = p->next;
        p->next = L->next;
        L->next = p;
        p= q; }     }
```

2.9.

```
void MergeLinkList ( LinkList &La, LinkList &Lb, LinkList &Lc) {
    LNode *pa = La->next;
    LNode *pb = Lb->next;
    LNode *q;
    Lc = La;
    Lc->next = NULL;
    while ( pa != NULL && pb != NULL) {
        if ( pa->data <= pb->data) {
            q= pa->next;
            pa->next = Lc->next;
            Lc->next = pa;
            pa = q;
        } else {
            q = pb->next;
            pb->next = Lc->next;
            Lc->next = pb;
            pb = q;
        } //end if-else     } //end while
    LNode *remaining = ( pa != NULL) ? pa : pb;
    while (remaining != NULL) {
        q= remaining->next;
        remaining->next = Lc->next;
        Lc->next = remaining;
        remaining = q; }     //end while
    delete Lb;
}
```

2.10.

```
void Delete_s_prior (LinkList &L) {
    LNode *p = s;
    LNode *q;
    while (p->next->next != s) p = p->next;
    q = p->next;
    p->next = s;          去除节点后前后接上
    delete q;
}
```

2.12.

```
void PartitionOddEven (LinkList &L) {
    if (L->next == NULL || L->next->next == NULL) return;
    LNode *p = L->next;
    LNode *q;
    LNode *head_even = new LNode;
    head_even->next = NULL;
    L->next = NULL;
    while (p) {
        q = p->next;
        if (p->data % 2 == 1) {
            p->next = L->next;
            L->next = p;
        } else {
            p->next = head_even->next;
            head_even->next = p; }
        p = q; }
    Lnode *tail_odd = L;
    while (tail_odd->next) tail_odd = tail_odd->next;
    tail_odd->next = head_even->next;
    delete head_even; }
```

2.3.

```
void InsertSqList ( SqList &La, ElemType x) {
    if (La.length ==La.listsize)     Increment (La);
    int i=La.length-1;
    while (i>=0 && La.elem[i] > x) {
        La.elem[i+1] = La.elem[i];
        i--; }
    La.elem[i+1] = x;
    La.length ++;     }
```

2.5.

```
void ReverseSqList ( SqList &L ) {
    if (L.length <=1)   return;
    int mid = L.length /2;
    for (int i=0; i<mid ; i++) {
        ElemType temp = L.elem[i];
        L.elem[i] = L.elem[L.length-1-i];
        L.elem[L.length-1-i] = temp; }
```

2.6.

```
void ReverseLinkList_WithHeader ( LinkList &L) {
    if (L->next == NULL || L->next->next ==NULL)     return;
    LNode *p = L->next;
    LNode *q;
    L->next = NULL;
    while (p != NULL) {
        q = p->next;
        p->next = L->next;
        L->next = p;
        p= q; }     }
```

2.9.

```
void MergeLinkList ( LinkList &La, LinkList &Lb, LinkList &Lc) {
    LNode *pa = La->next;
    LNode *pb = Lb->next;
    LNode *q;
    Lc = La;
    Lc->next = NULL;
    while ( pa != NULL && pb != NULL) {
        if ( pa->data <= pb->data) {
            q= pa->next;
            pa->next = Lc->next;
            Lc->next = pa;
            pa = q;
        } else {
            q = pb->next;
            pb->next = Lc->next;
            Lc->next = pb;
            pb = q;
        } //end if-else     } //end while
    LNode *remaining = ( pa != NULL) ? pa : pb;
    while (remaining != NULL) {
        q= remaining->next;
        remaining->next = Lc->next;
        Lc->next = remaining;
        remaining = q; }     //end while
    delete Lb;
}
```

2.10.

```
void Delete_s_prior (LinkList &L) {
    LNode *p = s;
    LNode *q;
    while (p->next->next != s) p = p->next;
    q = p->next;
    p->next = s;          去除节点后前后接上
    delete q;
}
```

2.12.

```
void PartitionOddEven (LinkList &L) {
    if (L->next == NULL || L->next->next == NULL) return;
    LNode *p = L->next;
    LNode *q;
    LNode *head_even = new LNode;
    head_even->next = NULL;
    L->next = NULL;
    while (p) {
        q = p->next;
        if (p->data % 2 == 1) {
            p->next = L->next;
            L->next = p;
        } else {
            p->next = head_even->next;
            head_even->next = p; }
        p = q; }
    Lnode *tail_odd = L;
    while (tail_odd->next) tail_odd = tail_odd->next;
    tail_odd->next = head_even->next;
    delete head_even; }
```

7.
```
bool match(char exp[]){
    int matchstat=1; InitStack(&S);
    ch=*exp++;
    while(ch!='#'&&matchstat){
        switch(ch){
            case '(':
            case '[':
            case '{': push(&S,ch); break;
            case ')': if(!Pop(&S,&e)&&e!='(')
                          matchstat=0; break;
            case ']': 类似上面
            case '}': 类似上面 }
        ch=*exp++; }
    if(matchstat&&StackEmpty(S)) return TRUE;
                          else return FALSE; }
```

转化成后缀表达式
```
void getsuffix(char exp[], char suffix[]){
    InitStack(S); push(S,'#');
    char *p=exp; k=0;
    while(!StackEmpty(S)){
        ch=*p++;
        if(!isoperator(ch)){suffix[k++]=ch; continue;}
        switch(ch){ case'(': push(S,ch); break;
                    case')': while(pop(S,c)&&c!='(')
                                 suffix[k++]=c;
                             break;
                    default: while(GetTop(S,c)&&preop(c,ch))
                                 {suffix[k++]=c; pop(S,c);}
                             if(ch!='#') push(S,ch);
                             break; }}
    suffix[k]='\0'; DestroyStack(S); }
```

8.
```
bool preop(char a, char b){
    int a_level, b_level;
    switch(a){ case'#': a_level=-1; break;
               case'(': a_level=0; break;
               case'+':
               case'-': a_level=1; break;
               case'*':
               case'/': a_level=2; break; } 闭括 switch(ch)
    return a_level>=b_level;
}
```

9.
```
int calculate(char suffix[]){
    char *p=suffix; InitStack(S);
    ch=*p++;
    while(ch!='#'){ if(!isoperator(ch)) push(S,ch);
                    else { pop(S,b); pop(S,a);
                           push(S,operate(a,ch,b)); }
                    ch=*p++; }
    pop(S,e);
    DestroyStack(S);
    return(e); }
```

10.
```
typedef struct LNode {
        ElemType data;
        struct LNode *next;
} LNode, * LinkQueueRear;
void InitQueue ( LinkQueueRear &rear) {
        rear -> next = rear; }
void EnQueue ( LinkQueueRear &rear, ElemType e) {
        LNode *p= new LNode;
        p-> data = e;
        p-> next = rear -> next
        rear ->next = p;
        rear = p;  }
bool DeQueue ( LinkQueueRear &rear, ElemType &e) {
        if (rear == rear -> next)  return FALSE;
        LNode *head = rear -> next;
        LNode *p = head -> next;
        e = p -> data;
        if ( p == rear)  rear = rear -> next;
        head -> next = p -> next;
        delete p;  return TRUE; }
```

11.
```
void EnQueue (SqQueue &Q, ElemType e) {
        if ( Q.length == Q.maxsize) Increment (Q)
        Q.rear = (Q.rear+1) % Q.maxsize;
        Q.elem [Q.rear] = e;
        Q.length ++; }
bool DeQueue (SqQueue &Q, ElemType &e ) {
        if (Q.length == 0 )  return FALSE;
        int front = (Q.rear - Q.length + 1 + Q.maxsize) % Q.maxsize;
        e= Q.elem [front];
        Q.length--;  return TRUE; }
```

队满：Q.length = Q.maxsize.

5.1

0,0,0,0; 0,0,0,1; 0,0,0,2; 0,0,1,0; 0,0,1,1; 0,0,1,2; 0,0,2,0; 0,0,2,1; 0,0,2,2;

0,1,0,0; 0,1,0,1; 0,1,0,2; 0,1,1,0; 0,1,1,1; 0,1,1,2; 0,1,2,0; 0,1,2,1; 0,1,2,2;

1,0,0,0; 1,0,0,1; 1,0,0,2; 1,0,1,0; 1,0,1,1; 1,0,1,2; 1,0,2,0; 1,0,2,1; 1,0,2,2;

1,1,0,0; 1,1,0,1; 1,1,0,2; 1,1,1,0; 1,1,1,1; 1,1,1,2; 1,1,2,0; 1,1,2,1; 1,1,2,2;

5.2

(1) $6 \times 6 \times 8 = 288$

(2) $1000 + 288 - 6 = 1282$

(3) $1000 + (2 \times 8 + 4) \times 6 = 1120$

(4) $1000 + (4 \times 6 + 2) \times 6 = 1156$

5.3

$$k = n + (n-1) + \ldots + (n-i+2) + (j-i+1) - 1$$

$$= -\frac{1}{2}i^2 + (n+\frac{1}{2})i + j - (n+1)$$

$$\Rightarrow f_1(i) = -\frac{1}{2}i^2 + (n+\frac{1}{2})i \;;\; f_2(j) = j \;;\; c = -(n+1)$$

5.4

$$k = 4 \times \left\lfloor \frac{i-1}{2} \right\rfloor + 2 \times \big((i-1)(mod2)\big) + (j-1)(mod2)$$

$$= 2(i-1) + (j-1)(mod2)$$

5.5

三元组	i	j	a_{ij}
	0	1	1
	0	4	5
	1	0	2
	1	1	3
	1	3	6
	3	1	4
	3	4	7

十字链表

rhead[0] → (0,1,1) →(right) (0,4,5) → NULL

rhead[1] → (1,0,2) → (1,1,3) → (1,3,6) → NULL

rhead[2] → NULL

rhead[3] → (3,1,4) → (3,4,7) → NULL

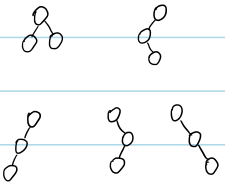
chead[0] → (1,0,2) →(down) NULL

chead[1] → (0,1,1) → (1,1,3) → (3,1,4) → NULL

chead[2] → NULL

chead[3] → (1,3,6) → NULL

chead[4] → (0,4,5) → (3,4,7) → NULL

6.1 树：  二叉树 

6.2 2个

6.3 (1) 所有结点左子树为空 / 空二叉树

(2) 所有结点右子树为空 / 空二叉树

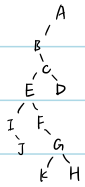
(3) 只有根结点 / 空二叉树

6.4 (1) 第h层：k^{h-1}

(2) $\lfloor \frac{i-2}{k} \rfloor + 1$

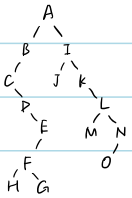
(3) $(i-1)k+1+j$

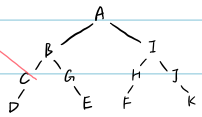
(4) $(i-1) \bmod k \neq 0$ ， $i+1$

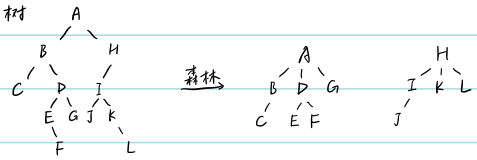
6.8 

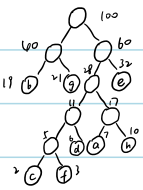
先根遍历：ABCEIJFGKHD

后根遍历：BIJEFKGHCDA

6.9 

6.11 

6.13 对应二叉树 

6.14 (1) 

(2)

a	1010
b	00
c	10000
d	1001
e	11
f	10001
g	01
h	1011

(3) WPL = 7×4 + 19×2 + 2×5 + 6×4 + 32×2 + 3×5 + 21×2 + 10×4 = 261

6.16 按单上结构体定义方式.

```
void swap (BiTree T) {
    if (!T) return;
    BiTree temp = T->lchild;
    T->lchild = T->rchild;
    T->rchild = temp;
    swap(T->lchild);
    swap(T->rchild); }
```

6.22

```
int TreeDepth (CSTree T) {
    int maxh = 0;
    if (!T) return 0;
    CSTree p; int h;
    for (p = T->firstchild; p; p = p->nextsibling) {
        h = TreeDepth (p);
        if (h > maxh) maxh = h; }
    return maxh+1; }
```

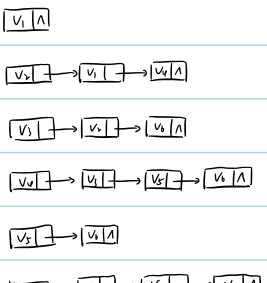
6.23

```
int Count (BiTree T, int k) {
    if (!root) return 0;
    if (k < 1) return 0;
    if (k = 1) return 1;
    return Count (T->lchild, k-1) + Count (T->rchild, k-1);
}
```

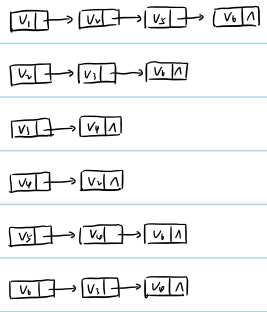
7.1 (1)

	V_1	V_2	V_3	V_4	V_5	V_6
V_1	0	0	0	0	0	0
V_2	1	0	0	1	0	0
V_3	0	1	0	0	0	1
V_4	0	0	1	0	1	1
V_5	1	0	0	0	0	0
V_6	1	1	0	0	1	0

(2)



(3)



(4) $\{V_2, V_3, V_4, V_6\}$, $\{V_1\}$, $\{V_5\}$

7.2

1: 7, 9 6: 1, 2

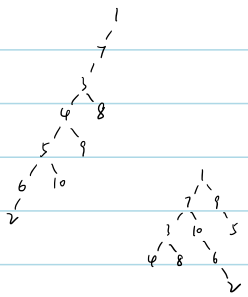
2: 3, 7 7: 3, 10

3: 4, 8 8: 1, 4, 9

4: 5, 9 9: 3, 7, 10

5: 6, 10 10: 1, 6

(1) DFS: 1, 7, 3, 4, 5, 6, 2, 10, 9, 8



(2) BFS: 1, 7, 9, 3, 10, 5, 4, 8, 6, 2

S	P[A]	P[B]	P[C]	P[D]	P[E]	P[F]	P[G]
A	/	A	A	A	/	/	/
A,C	/	A	A	A	C	C	/
A,C,F	/			F	C	C	F
A,C,F,E	/			F			F
A,C,F,E,D	/						D
A,C,F,E,D,G	/						D
A,C,F,E,D,G,B	/						D

7.3. (1)

	A	B	C	D	E	F	G	H
A	0	4	3					
B	4	0	5	5	9			
C	3	5	0	5				5
D		5	5	0	7	6	5	4
E		9		7	0	3		
F				6	3	0	2	
G				5		2	0	6
H			5	4			6	0

空白处为 ∞

A开始: A-C, A-B, B-D, D-H, D-G, G-F, F-E. $W_{min} = 26$

(2) A → B 4 → C 3 ∧

B → A 4 → C 5 → D 5 → E 9 ∧

C → A 3 → B 5 → D 5 → H 5 ∧

D → B 5 → C 5 → E 7 → F 6 → G 5 → H 4 ∧

E → B 9 → D 7 → F 3 ∧

F → D 6 → E 3 → G 2 ∧

G → D 5 → F 2 → H 6 ∧

H → C 5 → D 4 → G 6 ∧

Krustal: 2 F-G

3 A-C E-F

4 A-B D-H

5 B-C B-D C-D C-H D-G

6 D-F G-H

7 D-E

9 B-E

F-G A-C E-F A-B D-H ~~B-C~~ B-D ~~C-D~~ ~~C-H~~ D-G

⇒ F-G, A-C, E-F, A-B, D-H, B-D, D-G.

7.4. $V_1 < V_2$ $V_5 < V_6 < V_2 < V_3 < V_4$

3种 $V_1, V_5, V_6, V_2, V_3, V_4$

$V_5, V_1, V_6, V_2, V_3, V_4$

$V_5, V_6, V_1, V_2, V_3, V_4$.

7.5.

S	D[A]	D[B]	D[C]	D[D]	D[E]	D[F]	D[G]
A	0	15	2	12	∞	∞	∞
A,C	0	15	2	12	10	6	∞
A,C,F				11	10	6	16
A,C,F,E				11			16
A,C,F,E,D							14
A,C,F,E,D,G							14
A,C,F,E,D,G,B							14

A → C d=2 A → C → F → D d=11 D= (0, 15, 2, 11, 10, 6, 14)
A B C D E F G

A → C → F d=6 A → C → F → D → G d=14

A → C → E d=10 A → B d=15

7.8

```
void InDegree ( ALGraph G, int Indegree[ ]) {
    int i ;
    ArcNode *p ;
    for ( i=0; i < G. vexnum ; i++) Indegree[i]=0;
    for ( i=0; i< G. vexnum; i++) {
        p = G. vertices[i]. firstarc;
        while (p!= NULL) {
            Indegree [ p→ adjvex] ++;
            p = p→ nextarc ;
        } } }
```

7.9

```
bool DFSPath (ALGraph G, int v, int j, bool visited [ ]) {
    ArcNode *p;
    visited [v]= true;
    if (v==j) return true;
    p= G. vertices [v]. firstarc;
    while (p != NULL) {
        int w= p→ adjvex;
        if (! visited [w]) {
            if ( DFSPath ( G. w, j, visited)) {
                return true;
            }
        }
        p = p→ nextarc; }
    return false ; }
bool ExitPath ( ALGraph G, int i, int j) {
    bool visited [MAX_VERTEX_NUM];
    for (int k=0; k< G.vexnum ; k++)
        visited [k] = false;
    return DFSPath (G, i, j, visited) ;
}
```

7.10

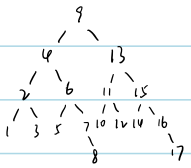
```
bool BFSPath ( ALGraph G, int i, int j) {
    bool visited [MAX_VERTEX_NUM];
    for (int k=0; k< G.vexnum ; k++)
        visited [k] = false;
    int v,w;
    ArcNode *p;
    InitQueue (&Q);
    visited [i] = true;
    EnQueue ( &Q, i);
    while (! QueueEmpty (Q)) {
        DeQueue ( &Q, &v);
        p = G. vertices [v]. firstarc;
        while (p != NULL) {
            w= p→ adjvex;
            if (w == j) return true;
            if (!visited [w]) { visited [w]= true;
                EnQueue ( &Q, w); }
            p=p→ nextarc;
        }
    }
    return false ; }
```

9.2 (1) 查 e: e>d e<f e=e d, f, e

(2) 查 f: f>d f=f d, f

(3) 查 g: g>d g>f g=g d, f, g

9.3



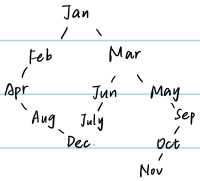
ASL成功 = $\frac{1\times1+2\times2+4\times3+8\times4+2\times5}{17} = \frac{59}{17}$

(-∞, 1) (1, 2) (2, 3), ---, (16, 17) (17, +∞)

ASL失败 = $\frac{14\times4+4\times5}{18} = \frac{38}{9}$.

9.4 (1) ASL = $0.1\times1+0.25\times2+\cdots+0.07\times12 = 5.43$

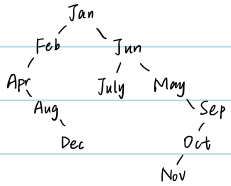
(2)



(3) ASL = $0.1\times1+0.25\times2+0.05\times2+0.13\times3+0.01\times3+0.06\times3$

$+0.11\times4+0.07\times4+0.02\times4+0.03\times5+0.1\times6+0.07\times5 = 3.2$

(4).



9.5

Key	10	25	33	19	06	49	37	76	60
H(key)	10	3	0	8	6	5	4	10	5

(1).

i	10	3	0	8	6	5	4	1	7
次数	1	1	1	1	1	1	1	3	3
key	10	25	33	19	06	49	37	76	60.

即

i	0	1	2	3	4	5	6	7	8	9	10
Key	33	76	空	25	37	49	06	60	19	空	10.

ASL成功 = $\frac{7\times1+2\times3}{9} = \frac{13}{9}$

ASL失败 = $\frac{3+2+1+7+6+\cdots+1+4}{11} = \frac{38}{11}$.

(2)

i	10	3	0	8	6	5	4	2	9
次数	1	1	1	1	1	1	1	6	3
key	10	25	33	19	06	49	37	76	60

即

i	0	1	2	3	4	5	6	7	8	9	10
Key	33	空	76	25	37	49	06	空	19	76	10

ASL成功 = $\frac{1\times7+6+3}{9} = \frac{16}{9}$

(3)

i	0	1	2	3	4	5	6	7	8	9	10
Key	33	空	空	25	37	49→60	06	空	19	空	10→76

ASL成功 = $\frac{1\times7+2+2}{9} = \frac{11}{9}$

9.7

```
bool Order(BiTree T, int *pre, bool *hasPre){
    if (T==NULL) return true;
    if (!Order(T->lchild, pre, hasPre)) return false;
    if (*hasPre && T->key<=*pre) return false;
    *pre = T->key;
    *hasPre = true;
    return Order(T->rchild, pre, hasPre);
}
bool IsBST(BiTree T){
    int pre=0;
    bool hasPre=false;
    return Order(T, &pre, &hasPre);
}
```

10.1 5 1 6 0 9 2 8 3 7 4

(1) 直接插入排序

(1 5) 6 0 9 2 8 3 7 4
(1 5 6) 0 9 2 8 3 7 4
(0 1 5 6) 9 2 8 3 7 4
(0 1 5 6 9) 2 8 3 7 4
(0 1 2 5 6 9) 8 3 7 4
(0 1 2 5 6 8 9) 3 7 4
(0 1 2 3 5 6 8 9) 7 4
(0 1 2 3 5 6 7 8 9) 4
0 1 2 3 4 5 6 7 8 9

(2) 希尔排序

2 1 3 0 4 5 8 6 7 9
0 1 3 2 4 5 8 6 7 9
0 1 2 3 4 5 6 7 8 9

(3) 快速排序

pivot=5 4 1 3 0 2 5 8 9 7 6
4,8 2 1 3 0 4 5 6 7 8 9
2,6 0 1 2 3 4 5 6 7 8 9

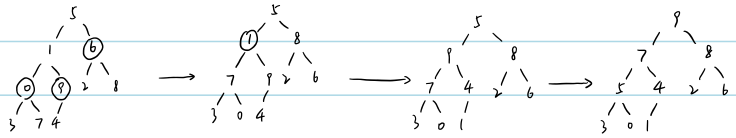
(4) 冒泡排序

1 5 0 6 2 8 3 7 4 9
1 0 5 2 6 3 7 4 8 9
0 1 2 5 3 6 4 7 8 9
0 1 2 3 5 4 6 7 8 9
0 1 2 3 4 5 6 7 8 9

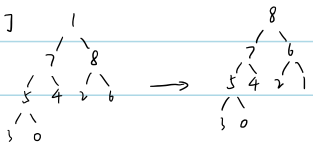
(5) 归并排序

(1 5) (0 6) (2 9) (3 8) (4 7)
(0 1 5 6) (2 3 8 9) (4 7)
(0 1 2 3 5 6 8 9) (4 7)
0 1 2 3 4 5 6 7 8 9

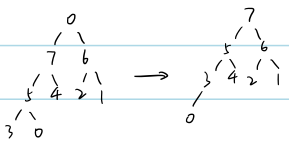
(6) 建堆



排序 [9]

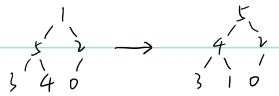


[8,9]

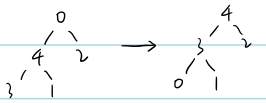


[7,8,9]

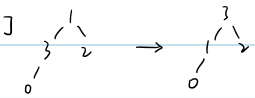
[6,7,8,9]



[5,6,7,8,9]

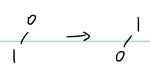


[4,5,6,7,8,9]



[3,4,5,6,7,8,9]

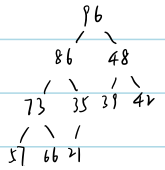
[2,3,4,5,6,7,8,9]



[1,2,3,4,5,6,7,8,9] 0

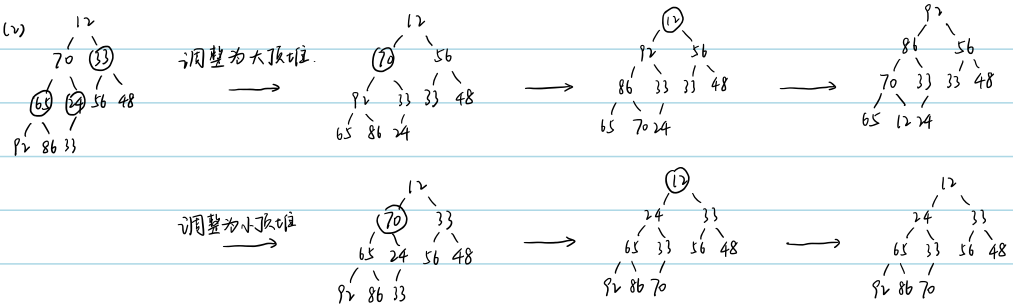
[0,1,2,3,4,5,6,7,8,9]

10.3 (1)



是大顶堆

(2)



10.5

```
void SelectSort (LinkList L) {
        LNode *sortedHead = NULL;
        LNode *sortedTail = NULL;
        while (L->next != NULL) {
                LNode *minPre = L;
                LNode *pre = L;
                while (pre->next != NULL) {
                        if (pre->next->key < minPre->next->key)
                                minPre = pre;
                        pre = pre->next; }
                LNode *min = minPre->next;
                minPre->next = min->next;
                min->next = NULL;
                SortedTail->next = min;
                SortedTail = min;    }
        L->next = sortedHead;
}
```